

qtcling 1.2.0 利用者ガイド

(株) SRA

2026-08-16

qtcling 1.2.0 利用者ガイド

qtcling とは

qtcling は、Cling の C++ REPL から Qt を対話的に使うための環境です。

通常のビルドを待たずに、REPL で Qt object や widget を作り、表示し、signal/slot や property の動きを確認することを目的にしています。

現在の位置づけ

1.2.0はsource code releaseです。

1.2.0のユーザー向け対象は、macOS、Linux、Windows 11 ARM64ネイティブ版の 端末版 `qtcling interactive`です。

1.1.0でmacOS/Linux版をリリースしました。1.2.0ではWindows 11 ARM64ネイティブ版を 正式に追加します。

macOS では、端末版 `qtcling` で QtCore / QtGui / QtWidgets と event callback が使えることを確認しています。

Linux では、端末版 `qtcling` の `--show-config` / `--check` と、Qt application の初期化を確認しています。

Windows 11 ARM64ネイティブ版では、REPL編集、履歴、Qt event、widget、signal/slot、Qt message出力を確認しています。詳細は [docs/windows-native-arm64.md](https://docs.qtcling.org/windows-native-arm64.md) を参照してください。

`qtcling-gui`、`qtcling-project`、`qtcling.toml`、Qt Creator plugin構想は 1.2.0のユーザー向け対象外です。

macOS クイックスタート

確認済みの基本環境変数です。

```
export QTCLING_QT_ROOT=/usr/local/qt/Qt/6.11.1/macos
export QTCLING_CLING_ROOT="$PWD/build"
```

```
export QTCLING_STARTUP_FILE="$PWD/src/qtgui.cpp"
```

`/usr/local/qtcling` に hook 入り Cling をまだ install していない場合は、
`QTCLING_CLING_ROOT="$PWD/build"` を指定してください。

端末版を起動します。

```
qtcling
```

起動後、`qApp` が使えることを確認します。

```
qApp
```

`widget` を作って表示します。

```
#include <QPushButton>
auto button = new QPushButton("OK?");
button->show();
```

source code release zip から使う

配布 zip は source code release です。 `build/`、`cling/`、`llvm-project/`、インストール済み Qt は含めていません。

展開します。

```
unzip qtcling-1.2.0-source.zip
cd qtcling-1.2.0
```

インストール手順

`qtcling` を使うには、Qt のほかに、interactive patch を適用した Cling が必要です。

依存 library

Qtの既定installation pathは `bin/qtcling-defaults.sh` で指定できます。永続的に設定する場合は、build前に同fileの `QTCLING_DEFAULT_QT_ROOT` をQt kitの 完全な pathへ変更します。現在のshellだけで上書きする場合は、環境変数をexportします。

```
export QTCLING_DEFAULT_QT_ROOT=/usr/local/qt/Qt/6.11.1/macos
./build.sh
```

`bin/qtcling-defaults.sh` はbuildと起動wrapperの両方で読み込まれ、install時にも install先へコピーされます。環境変数 `QTCLING_QT_ROOT` は個別起動用の完全なkit pathで、`QTCLING_DEFAULT_QT_ROOT` より優先されます。

Linuxでは、REPL入力待ち中にQt eventを処理するため、LLVMのlibedit LineEditor supportが必要です。Ubuntuではbuild前に `libedit-dev` を入れてください。

```
sudo apt install libedit-dev
```

`libedit-dev` がない場合、build は進むことがありますが、REPL 入力待ち中に Qt event が処理されず、button click などが Enter 後にまとめて反応することがあります。

`build.sh` は `LLVM_ENABLE_LIBEDIT=OFF` を検出すると警告します。

macOS では Xcode SDK に含まれる libedit を使います。

`libzstd` は任意です。qtcling interactive の通常利用には必要ありません。

LLVM/CMake が環境にある zstd を見つける場合がありますが、zstd support の有無は通常の Qt REPL 利用には影響しません。

配布物の `build.sh` は次を行います。

1. `cling/` がなければ `root-project/cling` をcloneし、revision `af630d98` へ固定する。
2. `llvm-project/` がなければ `root-project/llvm-project` をcloneし、revision `7c49650f1446` へ固定する。
3. 既存checkoutのrevisionとtracked changesを検証する。

4. `patch/qtcling-interactive/` の必須パッチを検証して適用する。

5. `build/` でClingをRelease buildする。

必須パッチが適用できず、既に適用済みでもない場合、`build.sh` はそこで停止します。

実行例:

```
./build.sh
```

`build.sh` はbuildだけを行い、installは行いません。

install して使う

`build.sh` の完了後、repository rootで次を実行します。

```
sudo ./install.sh
```

`install.sh` はClingとqtcling launcher、platform wrapper、既定path設定、startup file、`QTCLING_VERSION` を、CMakeで指定したprefixヘインストールします。既定は `/usr/local/qtcling` です。

この場合、macOS wrapper は既定で `/usr/local/qtcling/bin/cling` を使うため、`QTCLING_CLING_ROOT` は通常不要です。

macOS では Qt root を指定して確認します。

```
export QTCLING_QT_ROOT=/usr/local/qt/Qt/6.11.1/macos
bin/qtcling --check
bin/qtcling --show-config
bin/qtcling
```

install せず build tree のまま使う

`/usr/local/qtcling` に install しない場合は、`build tree` を `QTCLING_CLING_ROOT` で指定します。

```
export QTCLING_QT_ROOT=/usr/local/qt/Qt/6.11.1/macos
export QTCLING_CLING_ROOT="$PWD/build"
bin/qtcling --check
bin/qtcling --show-config
bin/qtcling
```

`$QTCLING_CLING_ROOT/bin/cling` が使われます。

手動で clone / patch / build する

`build.sh` を使わずに手動で進める場合は、次の順に実行します。

```
git clone https://github.com/root-project/cling.git
git -C cling checkout --detach af630d98
git clone https://github.com/root-project/llvm-project.git
git -C llvm-project checkout --detach 7c49650f1446
git apply patch/qtcling-interactive/0001-cling-add-periodic-callback-
api.patch
git apply patch/qtcling-interactive/0002-llvm-lineeditor-periodic-
callback.patch
git apply patch/qtcling-interactive/0003-cling-load-qtcling-startup-
file.patch
git apply patch/qtcling-interactive/0006-warn-when-libedit-is-disabled.patch
mkdir -p build
cd build
cmake -G Ninja \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=/usr/local/qtcling \
  -DLLVM_ENABLE_PROJECTS=clang \
  -DLLVM_EXTERNAL_PROJECTS=cling \
  -DLLVM_EXTERNAL_CLING_SOURCE_DIR=../cling \
  -DLLVM_BUILD_TOOLS=OFF \
  -DLLVM_TARGETS_TO_BUILD=host \
  ../llvm-project/llvm
cmake --build . -j 2
```

install する場合:

```
cd ..  
sudo ./install.sh
```

`install.sh` は CMake がbuild時に設定したinstall prefixへCling、qtcling launcher、platform wrapper、既定path設定、startup file、`QTCLING_VERSION` を配置します。既定のinstall prefixは `/usr/local/qtcling` です。

install せずに使う場合:

```
cd ..  
export QTCLING_CLING_ROOT="$PWD/build"
```

startup file

source code release の wrapper は、展開先ディレクトリを自動的に `QTCLING_DEV_ROOT` として扱います。

通常は `QTCLING_STARTUP_FILE` を明示しなくても、展開先の `src/qtgui.cpp` が使われます。

別の startup file を使う場合だけ指定します。

```
export QTCLING_STARTUP_FILE="$PWD/src/qtgui.cpp"
```

起動確認

macOS では、まず Qt root を指定します。

```
export QTCLING_QT_ROOT=/usr/local/qt/Qt/6.11.1/macos
```

installしたClingとlauncherを確認する場合は、install先を直接実行します。

```
/usr/local/qtcling/bin/qtcling --check  
/usr/local/qtcling/bin/qtcling --show-config  
/usr/local/qtcling/bin/qtcling
```

source treeの `bin/qtcling` は、`build/bin/cling` が存在すればinstall先よりbuild treeを 優先します。installせずbuild treeを使う場合、または選択を明示したい場合は、先に `QTCLING_CLING_ROOT` を指定します。

```
export QTCLING_CLING_ROOT="$PWD/build"
bin/qtcling --check
```

必要なら `PATH` に追加します。

```
export PATH="$PWD/bin:$PATH"
```

Windows 11 ARM64 ネイティブ版

Windows 11 ARM64ネイティブ版は1.2.0の正式対象です。実機では

`bin/qtcling.cmd` による起動、libeditの行編集と履歴、`qApp`、Qt widget、入力待ち中の Qt event、`QObject::connect`、`QDebug()` のコンソール出力を確認しています。

前提環境

次のARM64ネイティブ環境を用意します。x64版との混在を避けてください。

- Windows 11 ARM64
- Visual Studio 2022 Build Tools
 - Desktop development with C++
 - MSVC v143 C++ ARM64 build tools
 - Windows 11 SDK (確認済みは10.0.26100.0)
- CMake 3.21以降
- Git for Windows (Git Bashを含む)
- Python 3.13 ARM64
- Qt 6.11.1 `msvc2022_arm64` kit

確認済みの配置は次のとおりです。

```
Python C:\Users\<user>\AppData\Local\Programs\Python\Python313-arm64\python.exe
Qt      C:\Qt\6.11.1\msvc2022_arm64
```

コマンドが利用できることをPowerShellで確認します。

```
git --version
cmake --version
& "$env:LOCALAPPDATA\Programs\Python\Python313-arm64\python.exe" --version
Test-Path C:\Qt\6.11.1\msvc2022_arm64\include\QtCore
```

qtclingソースの用意

1.2.0 source archiveを展開し、Windowsネイティブ対応fileを含むqtclingのsource rootを用意します。ここで利用者が用意するのはqtcling自身のソースだけです。Git Bashで、取得または展開したqtcling開発ソースのroot directoryへ移動します。directory名は任意です。この作業環境では C:\Users\sugita\cling です。移動後、次のファイルがあることを確認してください。

```
cd /c/Users/sugita/clang

test -f build-win.sh
test -f build-libedit-win.ps1
test -f patch/libedit-windows-arm64.patch
test -f windows/CMakeLists.txt
test -f src/qtguiwin.cpp
test -f src/qtguiwin-startup.cpp
```

Cling、LLVM、portable libeditを利用者が手動で取得する必要はありません。これらの取得とrevision固定は、後続の build-win.sh と build-libedit-win.ps1 が自動的に行います。

```
Cling          af630d98
LLVM           7c49650f1446
portable libedit f3a98319002d0eca05e2a370a68b28a2afa6a822
```

既存checkoutが異なるrevisionにあり、tracked changesがある場合は、変更を保護するため スクリプトが処理を中止します。

libeditの先行ビルド・インストール

Clingを構成する前に、PowerShellでportable libeditをビルドして `libedit-win-arm64/` へインストールします。

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\build-libedit-win.ps1
```

このスクリプトはlibeditをcloneし、固定revisionをcheckoutして `patch/libedit-windows-arm64.patch` を適用した後、ARM64 Releaseとして ビルド・インストールします。完了後に必要なファイルを確認します。

```
Test-Path .\libedit-win-arm64\lib\edit.lib
Test-Path .\libedit-win-arm64\share\libedit\termcap
```

どちらも `True` になることを確認してからCling/LLVMへ進みます。

Cling/LLVMのビルド・インストール

Git Bashから次を実行します。

```
./build-win.sh
```

`build-win.sh` も安全のためClingの構成前に `build-libedit-win.ps1` を呼び出します。先行工程が完了している場合、libeditは増分ビルドになります。スクリプト全体の処理順は次のとおりです。

1. ClingとLLVMを必要に応じてcloneし、固定revisionを検証する。
2. ARM64 Pythonの配置を確認する。
3. portable libeditをビルドし、`libedit-win-arm64/` へインストールする。

4. 共通パッチ 0001 、 0002 、 0003 、 0006 とWindows用パッチ 0007 、 0008 、 0009 を検証して適用する。
5. インストール済みの libedit-win-arm64/ を指定してCling/LLVMを構成する。
6. Cling/LLVMをVisual Studio 2022 ARM64 Releaseでビルドする。
7. Release構成のClingを qtcling-win-arm64/ へインストールする。

12 GBメモリの確認環境に合わせ、Cling/LLVMの並列度は `-j 2` です。ビルドには 30 分以上かかる場合があります。途中で停止しても、同じコマンドで増分ビルドを 再開できます。

Clingのインストール結果を確認します。

```
Test-Path .\qtcling-win-arm64\bin\cling.exe
```

Qt callback DLLのビルドと配置

PowerShellでARM64 DLLをビルドします。

```
cmake -S .\windows -B .\build-qtcling-win-runtime `
  -G "Visual Studio 17 2022" -A ARM64 `
  -DCMAKE_PREFIX_PATH=C:\Qt\6.11.1\msvc2022_arm64

cmake --build .\build-qtcling-win-runtime --config Release -j 2

Copy-Item `
  .\build-qtcling-win-runtime\bin\Release\qtcling-win.dll `
  .\bin\qtcling-win.dll -Force
```

配置を確認します。

```
Test-Path .\bin\qtcling-win.dll
```

`qtcling-win.dll` はREPL入力待ち中にQt eventを処理し、Qt messageをコンソールへ 転送するARM64ネイティブDLLです。ソースツリーを別の場所へ移した場合も、DLLは `bin/qtcling.cmd` と同じ `bin` ディレクトリへ配置してください。

Windowsへのインストール

Cling/LLVMとlibeditのbuild完了後、PowerShellで次を実行します。

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\install-win.ps1
```

既定のinstall先は %LOCALAPPDATA%\qtcling です。 install-win.ps1 はQt callback DLLを ARM64 Releaseとして -j 2 でbuildし、次をinstall先へ配置します。

```
bin\cling.exe
bin\qtcling.cmd
bin\qtcling-win.dll
share\libedit\termcap
src\qtguiwin-startup.cpp
QTCLING_VERSION
```

install先を変更する場合はWindows形式のpathで指定します。

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\install-win.ps1 `
  -InstallDir C:\Tools\qtcling
```

Qtを標準位置以外へinstallしている場合は -QtRoot も指定します。

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\install-win.ps1 `
  -QtRoot D:\Qt\6.11.1\msvc2022_arm64
```

既定のinstall先から起動する場合は次を実行します。

```
& "$env:LOCALAPPDATA\qtcling\bin\qtcling.cmd"
```

起動とインストール結果の確認

Git Bashから起動します。

```
bin/qtcling.cmd
```

Qtを標準位置以外へインストールした場合は、起動前にWindows形式のpathを指定します。

```
export QTCLING_QT_ROOT='D:\Qt\6.11.1\msvc2022_arm64'  
bin/qtcling.cmd
```

起動に成功すると `[cling]$` プロンプトが表示されます。まず次を確認します。

```
qApp
```

`(QApplication *) 0x...` と表示されれば、startup fileとQt callback DLLの初期化は成功しています。

REPLでの確認例:

```
qApp  
#include <QPushButton>  
const auto b = new QPushButton("OK?");  
QObject::connect(b, &QPushButton::clicked,  
                 []() { qDebug() << "Clicked"; });  
b->show();
```

ボタンをクリックして `Clicked` がコンソールへ表示されること、上下キーで履歴を呼び出せること、左右キーで入力行を編集できることを確認します。終了は `.q` です。

生成先と再ビルド

libedit/	portable libedit source
build-libedit-win/	libedit build tree
libedit-win-arm64/	libedit install tree
cling/	Cling source
llvm-project/	LLVM/Clang source
build-win-arm64/	Cling/LLVM build tree
qtcling-win-arm64/	Cling install tree

```
build-qtcling-win-runtime/ Qt callback DLL build tree
bin/qtcling-win.dll        起動時に読み込むQt callback DLL
```

通常の再ビルドは、Git Bashで再び次を実行します。

```
./build-win.sh
```

Qt callback DLLだけを変更した場合は「Qt callback DLLのビルドと配置」の `cmake -build` と `Copy-Item` だけを再実行します。

よくある問題

- `ARM64 Python not found` : Python 3.13 ARM64の配置を確認します。現在の `build-win.sh` は `%LOCALAPPDATA%\Programs\Python\Python313-arm64\python.exe` を使います。
- `Qt ARM64 headers not found` : `QTCLING_QT_ROOT` が `msvc2022_arm64` kitを指すようにします。
- `cling.exe not found` : `./build-win.sh` のinstall段階まで完了したか確認します。
- `qtcling-win.dll` を読み込めない: DLLを `bin/` へコピーし、Qtの `bin` が launcherの `PATH` へ追加されていることを確認します。
- `.\version: error: invalid suffix '.0' on floating constant` : 古い `VERSION` fileがsource rootに残っています。Windowsでは標準C++ header `<version>` と衝突するため削除し、release versionには `QTCLING_VERSION` を使います。
- logo直後のアクセス違反: `patch/libedit-windows-arm64.patch` に含まれる `getline()` 修正を適用し、libeditとClingを再リンクします。
- Qt widgetは表示されるが反応しない: Windows用LineEditor callbackを含む最新の `0002-llvm-lineeditor-periodic-callback.patch` を適用して再ビルドします。

ビルド構成、固定revision、portable libeditの修正、Windows専用Qt callback DLL、トラブルシュートは [docs/windows-native-arm64.md](https://docs.sra-inc.com/windows-native-arm64.md) にまとめています。

配布物の内容

公開用source archiveに含まれる主要fileは次のとおりです。

- `build.sh` / `install.sh`
- `build-win.sh` / `build-libedit-win.ps1` / `install-win.ps1`
- `QTCLING_VERSION`
- `bin/qtcling`
- `bin/qtcling.cmd`
- `bin/iqtcling.macos` / `bin/cqtcling.macos`
- `bin/iqtcling.linux` / `bin/cqtcling.linux`
- `bin/qtcling-defaults.sh`
- `bin/run_moc`
- `bin/run_uic`
- `bin/run_rcc`
- `bin/run_all`
- `patch/qtcling-interactive.patch`
- `patch/qtcling-interactive/`
- `patch/libedit-windows-arm64.patch`
- `examples/`
- `src/`
- `windows/`

interactive patch を適用する

`qtcling` の interactive event loop 対応には、`cling/` と `llvm-project/` への patch が必要です。通常は `build.sh` が自動適用するため、以下は手動buildを行う場合だけです。

top-level checkout に `cling/` と `llvm-project/` がある状態で、配布物内の必須パッチを適用します。

```
git apply patch/qtcling-interactive/0001-cling-add-periodic-callback-  
api.patch  
git apply patch/qtcling-interactive/0002-llvm-lineeditor-periodic-  
callback.patch  
git apply patch/qtcling-interactive/0003-cling-load-qtcling-startup-  
file.patch  
git apply patch/qtcling-interactive/0006-warn-when-libedit-is-disabled.patch
```

統合パッチ `patch/qtcling-interactive.patch` は、この 4 つの必須パッチだけを連結したものです。

個別に確認したい場合は、`patch/qtcling-interactive/` の patch を参照してください。

- `0001-cling-add-periodic-callback-api.patch`
- `0002-llvm-lineeditor-periodic-callback.patch`
- `0003-cling-load-qtcling-startup-file.patch`
- `0006-warn-when-libedit-is-disabled.patch`

`0004-qtcling-wrapper-macos-linux-fixes.patch` と `0005-linux-wrapper-diagnostics.patch` は top-level wrapper 用の参考パッチです。最新の checkout では top-level wrapper 修正は Git 側に含まれているため、通常は適用しません。

端末版 qtcling

`qtcling` は端末上の Cling REPL を使います。

```
qtcling
```

macOS interactive 版では、`QTCLING_STARTUP_FILE` により `src/qtgui.cpp` を起動時に読み込みます。そのため、手動で `.L qtgui.cpp` を実行しなくても `QApplication` が準備されます。

`examples` を読み込む例です。

```
.I examples
.L button.cpp
```

`examples/button.cpp` は button click 時に `OK?` を出力します。REPL 入力待ち中でも Qt event が処理されることを確認できます。

GUI 版 qtcling-gui

`qtcling-gui` は、Qt GUI アプリ内に REPL 風の入力欄と出力欄を持つ prototype です。公開用 1.2.0 source archive には含まれません。開発 checkout でのみ利用できます。

```
qtcling-gui
```

端末の `LineEditor` / `libedit` に依存せず、Qt の `QApplication::exec()` 上で Cling を動かします。

確認例:

```
qApp
#include <QPushButton>
auto button = new QPushButton("OK?");
button->show()
```

現時点では 1 行入力のみです。履歴、補完、複数行入力などは未実装です。

環境変数

QTCLING_QT_ROOT

Qt installation root を指定します。

```
export QTCLING_QT_ROOT=/usr/local/qt/Qt/6.11.1/macos
```

macOS では、この下の `lib/*.framework` を使って include path と framework path を組み立てます。

QTCLING_CLING_ROOT

使用する Cling の root を指定します。

```
export QTCLING_CLING_ROOT="$PWD/build"
```

`$QTCLING_CLING_ROOT/bin/cling` が使われます。

QTCLING_STARTUP_FILE

qtcling 起動時に読み込む startup file を指定します。

```
export QTCLING_STARTUP_FILE="$PWD/src/qtgui.cpp"
```

この file が `QApplication` の準備と Qt event processing callback の設定を行います。

QTCLING_MODULES

使用する Qt module を指定します。

```
export QTCLING_MODULES="Core Gui Widgets Network"
```

デフォルトは `Core Gui Widgets` です。

例:

```
QTCLING_MODULES="Core Gui Widgets Network" qtcling
```

REPL で:

```
#include <QTcpSocket>
auto socket = new QTcpSocket;
socket
```

QTCLING_RESOURCE_DIR

`qtcling-gui` で Clang resource directory を明示する場合に使います。

```
export QTCLING_RESOURCE_DIR="$PWD/build/lib/clang/20"
```

通常は build-time default が使われます。

設定確認

macOS / Linux interactive版では、設定確認用optionがあります。

```
qtcling --version  
qtcling --show-config  
qtcling --check
```

`--version` は、配布物rootの `QTCLING_VERSION` fileに書かれたqtclingのrelease versionを表示します。

`--show-config` は、Qt root、module、framework path または library path、Cling executable、startup file、Cling args を表示します。

`--check` は、Qt root、Qt libraries、Cling executable、startup file の存在を確認します。

Qt module を追加する

追加 module を使う場合は `QTCLING_MODULES` を指定します。

```
QTCLING_MODULES="Core Gui Widgets Network" qtcling
```

macOSでは、指定されたmoduleからinclude pathとframework load設定を生成します。

Linuxでは、include path、`.so` preload、`-lQt6...` link引数を生成します。

qtcling-project

`qtcling-project` は、将来の `project helper` の最小 `prototype` です。公開用1.2.0 `source archive`には含まれません。開発`checkout`でのみ利用できます。

現時点では `explain` のみ実装されています。`project` 単位の自動生成や自動起動はまだ行いません。

```
qtcling-project explain examples/wiggly
```

表示する内容:

- `project directory`
- `qtcling.toml` の有無
- `.pro` の有無
- `sources`
- `headers`
- `forms`
- `resources`
- `translations`
- `Qt modules`
- `include dirs`
- `defines`
- `autogen outputs`
- `generated startup includes`
- `warnings`
- `unsupported features`

例:

```
qtcling-project explain examples/widgets/tutorials/notepad
```

`plugin project` は初期対象外です。検出した場合は `Unsupported` に表示します。

```
qtcling-project explain examples/cpp-advanced/solutions/sif
```

Qt generated files を作る

通常の Qt コードは、`moc` / `uic` / `rcc` が作るファイルに依存します。

たとえば `Q_OBJECT` を含む class は `moc` 出力が必要です。 `.ui` を使う code は `ui_*.h` が必要です。 `.qrc` を使う code は `qrc_*.cpp` が必要です。これらを生成しないと、ほとんどの Qt example はそのままでは実行できません。

`qtciling` には手動実行用の helper があります。

```
run_moc [files...]
run_uic [files...]
run_rcc [files...]
run_all
```

`run_moc` は、指定した C++ header/source のうち `Q_OBJECT` を含むものだけに `moc` を実行します。

入力	出力
<code>widget.h</code>	<code>moc_widget.cpp</code>
<code>widget.cpp</code>	<code>widget.moc</code>

`run_uic` は `.ui` から `ui_<basename>.h` を作ります。

入力	出力
<code>mainwindow.ui</code>	<code>ui_mainwindow.h</code>

`run_rcc` は `.qrc` から `qrc_<basename>.cpp` を作ります。

入力	出力
<code>resources.qrc</code>	<code>qrc_resources.cpp</code>

引数を省略すると、各 helper はカレントディレクトリの対象ファイル进行处理します。

例:

```
cd examples/wiggly
run_all
```

`run_all` は、カレントディレクトリで `run_moc` , `run_rcc` , `run_uic` を順に実行します。

```
run_all
```

`moc` / `uic` / `rcc` が PATH にない場合は、各 helper に明示できます。

```
run_moc --moc /path/to/moc widget.h
run_uic --uic /path/to/uic mainwindow.ui
run_rcc --rcc /path/to/rcc resources.qrc
```

翻訳ファイル

`TRANSLATIONS` や `.ts` を使う project では、通常の Qt build と同じく `.qm` が必要になる場合があります。

現時点の qtcling は、`qtcling-project explain` で translations を表示しますが、`lrelease` / `lupdate` / `lconvert` は実行しません。

- `lrelease` : `.ts` から `.qm` を作る処理です。将来の `qtcling-project prepare` では stale な `.qm` 生成対象にします。
- `lupdate` : `.ts` を更新する処理です。ユーザーの翻訳ファイルを書き換えるため、既定では自動実行しません。
- `lconvert` : 翻訳ファイル形式の変換などに使います。現時点では未対応です。

そのため翻訳を使う project では、必要に応じてユーザーが Qt の tool を直接実行してください。

```
lrelease *.ts
```

`.qm` を `.qrc` に含めている project では、`.qm` を先に更新してから resource を生成します。

```
lrelease *.ts
run_rcc *.qrc
```

または、`.qm` 更新後に通常どおり `run_all` を実行します。

```
lrelease *.ts
run_all
```

examples を動かす

現在の基本形は、必要な generated files を先に作り、対象ディレクトリを include path に追加し、`startup.cpp` を読み込んで `startup()` を呼ぶ方法です。

例:

```
cd examples/wiggly
run_all
cd ../../
```

```
qtcling
```

REPL で:

```
.I examples/wiggly
.L examples/wiggly/startup.cpp
startup()
```

今後は `qtcling-project prepare` や `qtcling --run .` で、`moc` / `uic` / `rcc` / `startup()` 生成を隠す予定です。

現在の制限

- 1.2.0はsource code releaseです。
- 1.2.0のユーザー向け対象はmacOS、Linux、Windows 11 ARM64ネイティブ版の端末版 `qtcling interactive`です。

- `qtcling-gui` はprototypeであり、1.2.0のユーザー向け対象外です。
- `qtcling-project` は `explain` のみであり、1.2.0のユーザー向け対象外です。
- `moc` / `uic` / `rcc` は helper で手動実行できますが、`project` 単位の自動実行は未実装です。
- `lrelease` / `lupdate` / `lconvert` は未実装です。
- `qtcling.toml` は読み取りの初期対応のみです。
- QML、plugin、deployment、D-Bus、Remote Objects、ShaderTools は初期対象外です。
- `plugin project` は REPL だけでは通常の Qt plugin として扱えません。

トラブルシュート

`qApp` が未定義になる

hook 入り Cling を使っているか確認してください。

```
export QTCLING_CLING_ROOT="$PWD/build"
qtcling
```

また、`QTCLING_STARTUP_FILE` が正しいか確認します。

```
qtcling --check
```

resource directory lib/clang/20 not found が出る

`qtcling-gui` で Clang resource directory が見つからない状態です。

```
export QTCLING_RESOURCE_DIR="$PWD/build/lib/clang/20"
qtcling-gui
```

Qt module の header が見つからない

`QTCLING_MODULES` に module を追加してください。

```
QTCLING_MODULES="Core Gui Widgets Network" qtcling
```

plugin example が動かない

plugin は初期対象外です。 `qtcling-project explain` で unsupported として表示される場合、その example は通常の autogen workflow では扱いません。